

HW9

Group 12 - Shir, Omer & Ido

16/06/26

Group:

Group members (Name, ID, email)

- Ido Ravid, 207198177, ido.ravid@mail.huji.ac.il
 - Omer Sutovsky, 318735107, omer.sutovsky@mail.huji.ac.il
 - Shir Tsarfaty, 207718008, shir.tsarfaty@mail.huji.ac.il
-

Paths and Data

```
# Ensure a CRAN mirror is set so install.packages() works on a fresh machine
if (is.null(getOption("repos")) || getOption("repos")["CRAN"] == "@CRAN@") {
  options(repos = c(CRAN = "https://cloud.r-project.org"))
}

if (!require('data.table')) { install.packages('data.table'); library('data.table') }
if (!require('splines'))      { install.packages('splines');   library('splines') }
if (!require('MASS'))         { install.packages('MASS');     library('MASS') }

source("../utils.R")
source("../lab5/lab5_utils.R")

healthy_file <- "../lab2/data/TCGA-13-0723-10B_lib2_all_chr1.forward"
tumor_file  <- "../lab7/data/TCGA-13-0723-01A_lib2_all_chr1.forward"
chr1_file   <- "../lab1/data/chr1_str.rda"

healthy_reads <- fread(healthy_file, col.names = c("Chr", "Loc", "Length"))
tumor_reads  <- fread(tumor_file,   col.names = c("Chr", "Loc", "Length"))
load(chr1_file)

BIN <- 2500

add_pos <- function(df, beg) {
  df$pos <- beg + (seq_len(nrow(df)) - 1) * BIN + BIN / 2
  df
}

# Two arms around centromere (120-130 Mb excluded), both samples
df_h <- rbind(
  add_pos(bin_data(healthy_reads, chr1, 5000001L, 119990000L, BIN), 5000001L),
  add_pos(bin_data(healthy_reads, chr1, 130000001L, 242000000L, BIN), 130000001L)
)
```

```

df_t <- rbind(
  add_pos(bin_data(tumor_reads, chr1, 5000001L, 119990000L, BIN), 5000001L),
  add_pos(bin_data(tumor_reads, chr1, 130000001L, 242000000L, BIN), 130000001L)
)

# Per-sample GC spline + plug-in estimator (lab 8 method b)
eps <- 1
gc_correct <- function(df) {
  nz <- df$reads > 0
  fit <- fit_gc_spline(df[nz, ])
  fhat <- pmax(predict(fit, data.frame(gc = df$gc)), 0)
  raw <- (df$reads + eps) / (fhat + eps)
  raw / median(raw[nz])
}
df_h$cn <- gc_correct(df_h)

## Warning in bs(gc, degree = 3L, knots = c(`25%` = 0.3684, `50%` = 0.4064, : some
## 'x' values beyond boundary knots may cause ill-conditioned bases

df_t$cn <- gc_correct(df_t)

## Warning in bs(gc, degree = 3L, knots = c(`25%` = 0.3684, `50%` = 0.4064, : some
## 'x' values beyond boundary knots may cause ill-conditioned bases

# Two-sample estimator (lab 8 method d): GC-corrected tumor / GC-corrected healthy
df_t$cn_2s <- df_t$cn / df_h$cn

nz_h <- df_h$reads > 0
nz_t <- df_t$reads > 0
nz <- nz_h & nz_t # bins with reads in both samples

# alias so rest of code uses df / df$cn referring to working data frame
df <- df_t

cat(sprintf("Total bins: %d | both non-zero: %d\n", nrow(df), sum(nz)))

## Total bins: 90796 | both non-zero: 85010

```

Part A – Test Statistic

We need a statistic T that summarises whether a genomic region has more or fewer reads than expected under copy number 1, while being **resistant to single noisy bins** – meaning one extreme bin should not be able to drive the entire region to significance on its own.

We use the **two-sample GC-corrected estimator** (Lab 8 method d): $\hat{a}_k = \hat{a}_k^{\text{tumor}} / \hat{a}_k^{\text{healthy}}$, where each sample is corrected by its own GC spline and median-normalised. Dividing cancels shared GC bias and library-specific technical effects, leaving a cleaner copy-number signal than single-sample correction alone.

We then take $\log_2(\hat{a}_k)$: under H_0 (CN = 1) this is centred at 0, gains are positive, losses are negative, and the $\log_2$ scale has a natural interpretation (a value of 1 means a doubling; -1 means a halving).

We considered three candidates, all computed as a rolling aggregate over a window of $W = 11$ bins (≈ 27.5 kb):

Statistic	Formula	Notes
S1: plain $\log_2$	$\log_2(\hat{a}_k)$ – per bin, no aggregation	Baseline
S2: trimmed mean	$\log_2(\hat{a})_{10\%}$ over window	Hard-discards top/bottom 10% of bins
S3: Huber location	$\hat{\mu}_{\text{Huber}}(\log_2(\hat{a}), k = 1.345)$ over window	Continuously downweights outlier bins

We evaluated all three against the empirical null (Part B). S2 and S3 produce noticeably narrower, more compressed null distributions when combined with the local-stability negative control filter: the filter already removes noisy bins, so a further robust aggregation over-compresses the tails. S1 – plain $\log_2(\hat{a}_k)$ – keeps the most natural spread. We carry all three forward and defer the final choice to Part B, where a calibration test on held-out control bins selects between them; the “resistance to single noisy bins” requirement is in any case addressed by the negative control selection (Part B) rather than by the statistic alone.

```

W      <- 11
HALF   <- (W - 1) / 2

log2_cn <- log2(pmax(df$cn_2s, 1e-6))  # log2 of two-sample a-hat

# Rolling window helper (kept for S2/S3 comparison plots)
roll_stat <- function(x, fun) {
  out <- rep(NA_real_, length(x))
  for (i in seq_along(x)) {
    idx <- max(1, i - HALF) : min(length(x), i + HALF)
    sub <- x[idx][nz[idx]]
    if (length(sub) >= 5) out[i] <- fun(sub)
  }
  out
}

df$S1 <- log2_cn
df$S2 <- roll_stat(log2_cn, function(x) mean(x, trim = 0.10))
df$S3 <- roll_stat(log2_cn, function(x) huber(x, k = 1.345)$mu)

```

Part B – Negative Control Regions and Null Distribution

How we select the negative controls. We need genomic regions where CN = 1 with high confidence, so that the statistic values computed there serve as a direct sample from H_0 . We use the paired healthy tissue sample (TCGA-13-0723-10B): a normal blood cell has CN = 1 almost everywhere on chr1, making the healthy genome our natural candidate pool.

We apply the local stability filter (**Strategy B**) directly on the **two-sample ratio** $\log_2(\hat{a}_k)$: a bin is kept as a negative control only if its 11-bin neighbourhood simultaneously has a median near 0 **and** a low local SD. We set the two cut-offs in interpretable linear-ratio units and then convert to $\log_2$: the local median CN must stay within $\pm 10\%$ of the no-change baseline ($|\tilde{T}| \leq \log_2 1.10 \approx 0.137$), and the local scatter must stay under $\pm 15\%$ ($\text{SD} \leq \log_2 1.15 \approx 0.202$). Filtering on the ratio rather than the healthy sample alone is the correct choice: H_0 is “no tumor-specific CN change” (ratio = 1), not “healthy CN = 1 absolutely.” This also handles germline CNVs correctly – if both samples carry a duplication at some locus, the ratio is still 1 and the bin is a valid negative control. Bins at tumor CN-event boundaries are excluded because their neighbours reveal the disruption even if the bin itself lands near 1. Approximately 37% of non-zero bins pass this filter, giving ~31,000 control bins across both chromosome arms.

How we model H_0 . The $\log_2(\hat{a}_k)$ values of the selected control bins are a direct sample from the distribution of the statistic when $CN = 1$ – we use them directly as the empirical null, without assuming any parametric form. The histograms and QQ plots below characterise this distribution for each of the three candidate statistics.

How p-values are derived (Part C). For any test bin k with statistic value T_k , the two-sided empirical p-value is the fraction of null values at least as extreme:

$$p_k = \frac{|\{j \in \text{ctrl} : |T_j| \geq |T_k|\}|}{|\text{ctrl}|}$$

With 31,000 control bins this resolves p-values down to $\approx 3 \times 10^{-5}$, sufficient for testing at $\alpha = 0.01$ after multiple testing correction. No normality assumption is required.

```
# Stability filter runs on the two-sample ratio (log2_cn = log2(tumor/healthy)).
# H0 is "no tumor-specific CN change" (ratio = 1), so the filter should assess
# whether the ratio is locally stable near 0 -- not whether the healthy alone is.
# This also correctly handles germline CNVs: if both samples have CN=2, ratio=1
# and the bin is still a valid negative control.
# Thresholds set in interpretable linear-ratio units, then converted to log2:
# median within +/-10% of the no-change baseline; local scatter under +/-15%.
med_thresh <- log2(1.10) # ~0.137
sd_thresh  <- log2(1.15) # ~0.202

win_med_log <- roll_stat(log2_cn, median)
win_sd_log  <- roll_stat(log2_cn, sd)
stratB <- nz & !is.na(win_med_log) & !is.na(win_sd_log) &
  abs(win_med_log) <= med_thresh & win_sd_log <= sd_thresh

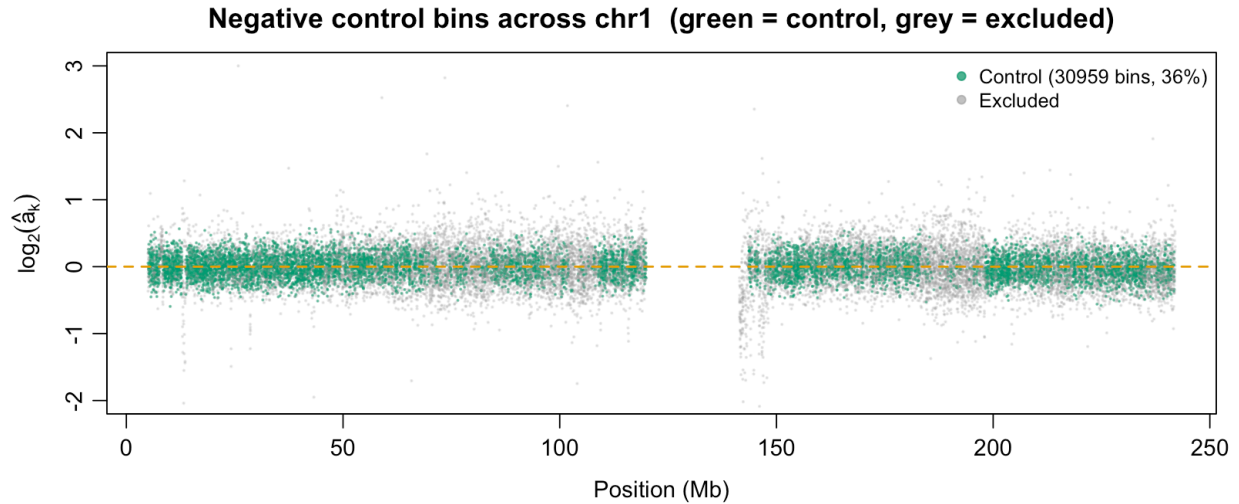
cat(sprintf("Strategy B (local stability): %d bins (%.0f%% of non-zero)\n",
  sum(stratB), 100 * sum(stratB) / sum(nz)))
```

```
## Strategy B (local stability): 30959 bins (36% of non-zero)
```

```
# Chromosome-wide scatter: control bins highlighted, excluded in grey
set.seed(42)
idx_all <- which(nz)
s <- sample(idx_all, min(25000, length(idx_all)))
x_mb <- df$pos[s] / 1e6
y_cn <- pmin(log2_cn[s], 3)

col_vec <- ifelse(stratB[s],
  adjustcolor("#009E73", 0.5),
  adjustcolor("#999999", 0.2))

par(mar = c(4.5, 4, 2.5, 1), mgp = c(2.2, 0.7, 0))
plot(x_mb, y_cn, pch = 20, cex = 0.25, col = col_vec,
  xlab = "Position (Mb)", ylab = expression(log[2](hat(a)[k])),
  main = "Negative control bins across chr1 (green = control, grey = excluded)",
  ylim = c(-2, 3))
abline(h = 0, col = "#E69F00", lwd = 1.5, lty = 2)
legend("topright", bty = "n", cex = 0.85,
  legend = c(sprintf("Control (%d bins, %.0f%%)",
    sum(stratB), 100 * sum(stratB) / sum(nz)), "Excluded"),
  col = c(adjustcolor("#009E73", 0.8), adjustcolor("#999999", 0.6)),
  pch = c(20, pt.cex = 1.2))
```



```

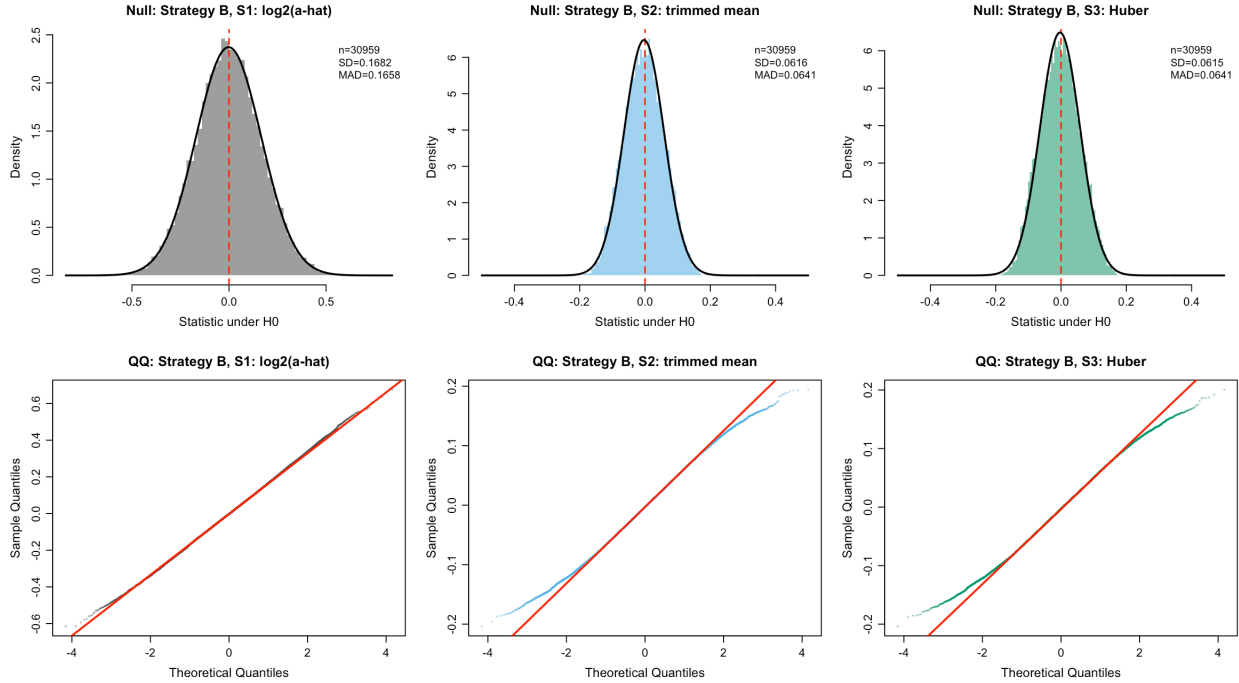
stat_list_b <- list(S1 = df$S1, S2 = df$S2, S3 = df$S3)
stat_labs_b <- c("S1: log2(a-hat)", "S2: trimmed mean", "S3: Huber")
cols_b      <- c("#555555", "#56B4E9", "#009E73")

par(mfrow = c(2, 3), mar = c(4.5, 4, 3, 1), mgp = c(2.4, 0.8, 0),
    cex.main = 1.3, cex.lab = 1.2, cex.axis = 1.1)

for (ti in 1:3) {
  mask <- stratB & !is.na(stat_list_b[[ti]])
  v    <- stat_list_b[[ti]][mask]
  # xlim: 5 SDs each side so shape is always visible, floor at +/-0.5
  hw   <- max(5 * sd(v), 0.5)
  xlim <- c(-hw, hw)
  xseq <- seq(xlim[1], xlim[2], length.out = 300)
  h    <- hist(v, breaks = 80, plot = FALSE)
  plot(h, freq = FALSE, col = adjustcolor(cols_b[ti], 0.6), border = NA,
       main = sprintf("Null: Strategy B, %s", stat_labs_b[ti]),
       xlab = "Statistic under H0", xlim = xlim)
  abline(v = 0, col = "red", lwd = 1.5, lty = 2)
  lines(xseq, dnorm(xseq, mean(v), sd(v)), col = "black", lwd = 2)
  legend("topright", bty = "n", cex = 1.0,
        legend = sprintf("n=%d\nSD=%.4f\nMAD=%.4f", length(v), sd(v), mad(v)))
}

for (ti in 1:3) {
  mask <- stratB & !is.na(stat_list_b[[ti]])
  v    <- stat_list_b[[ti]][mask]
  qqnorm(v, pch = 20, cex = 0.3, col = adjustcolor(cols_b[ti], 0.4),
        main = sprintf("QQ: Strategy B, %s", stat_labs_b[ti]))
  qqline(v, col = "red", lwd = 2)
}

```



The histograms are shown with a fitted normal overlay (black line) for reference, and the QQ plots assess normality directly. S2 and S3 are visibly narrower and more compressed than S1, a consequence of applying robust aggregation on top of bins already filtered for stability. But comparing shapes to a normal curve is only superficial – what actually matters is whether each statistic produces **calibrated** p-values. We test this directly below.

Validating calibration (split-half)

A statistic is only usable if, on data that truly follows H_0 , its p-values are **Uniform(0,1)** – equivalently, the realised false-positive rate at level α equals α . We can check this with the control bins alone, no labelled events required:

1. Split the control bins into **interleaved genomic blocks** (block size 100 bins $\gg$ window $W = 11$, so the two sets never share a window). Even blocks build the empirical null; odd blocks are held out as “test” bins.
2. The held-out bins are themselves $CN = 1$, so under a correctly calibrated test their two-sided empirical p-values must be uniform.
3. We measure the deviation from uniform (Kolmogorov–Smirnov D) and the realised false-positive rate at $\alpha = 0.01$ and 0.05 .

A statistic whose null is artificially narrow (S2/S3) will make ordinary $CN = 1$ fluctuations look significant, inflating the false-positive rate above α – which disqualifies it regardless of how its histogram looks.

```
# Interleaved-block split so null-building and test bins never share a window
ctrl_idx <- which(stratB)
blk      <- 100                                # block size (bins) >> W = 11
block_id <- ctrl_idx %/% blk
null_ix  <- ctrl_idx[block_id %% 2 == 0] # even blocks -> build null
test_ix  <- ctrl_idx[block_id %% 2 == 1] # odd blocks -> held-out CN=1 tests

# Add-one two-sided empirical p-value (never zero, valid by construction)
emp_p2 <- function(t_test, t_null) {
  n <- length(t_null)
```

```

  at <- abs(t_test)
  vapply(at, function(a) (1 + sum(abs(t_null) >= a)) / (n + 1), numeric(1))
}

p_lists <- list()
calib_rows <- list()
for (ti in seq_along(stat_list_b)) {
  v_null <- stat_list_b[[ti]][null_ix]; v_null <- v_null[is.finite(v_null)]
  v_test <- stat_list_b[[ti]][test_ix]; v_test <- v_test[is.finite(v_test)]
  m <- median(v_null) # H0 centre, estimated from the null
  p <- emp_p2(v_test - m, v_null - m)
  p_lists[[ti]] <- p
  calib_rows[[ti]] <- data.frame(
    Statistic = names(stat_list_b)[ti],
    n_test    = length(p),
    FPR_0.01  = round(mean(p < 0.01), 4),
    FPR_0.05  = round(mean(p < 0.05), 4),
    KS_D      = round(suppressWarnings(ks.test(p, "punif")$statistic), 4)
  )
}
calib_tbl <- do.call(rbind, calib_rows)
rownames(calib_tbl) <- NULL
knitr::kable(calib_tbl,
  col.names = c("Statistic", "n (test)", "FPR @ 0.01", "FPR @ 0.05", "KS D"),
  caption = "Calibration on held-out CN=1 bins. Well-calibrated: FPR approx
  ↪ alpha, KS D small.")

```

Table 1: Calibration on held-out CN=1 bins. Well-calibrated: FPR approx alpha, KS D small.

Statistic	n (test)	FPR @ 0.01	FPR @ 0.05	KS D
S1	15633	0.0086	0.0496	0.0088
S2	15633	0.0077	0.0430	0.0155
S3	15633	0.0090	0.0440	0.0137

```

cols_b <- c("#555555", "#56B4E9", "#009E73")
par(mfrow = c(1, 2), mar = c(4.5, 4.5, 3, 1), mgp = c(2.6, 0.8, 0),
  cex.main = 1.2, cex.lab = 1.2, cex.axis = 1.1)

# Full P-P plot: empirical CDF of p-values vs uniform diagonal
plot(NA, xlim = c(0, 1), ylim = c(0, 1),
  xlab = "Nominal p", ylab = "Observed fraction <= p",
  main = "Calibration (P-P plot): full range")
abline(0, 1, col = "grey50", lwd = 2, lty = 2)
for (ti in 1:3) {
  ps <- sort(p_lists[[ti]])
  lines(ps, (seq_along(ps)) / length(ps), col = cols_b[ti], lwd = 2.5)
}
legend("bottomright", bty = "n", cex = 1.1, lwd = 2.5, col = cols_b,
  legend = names(stat_list_b))

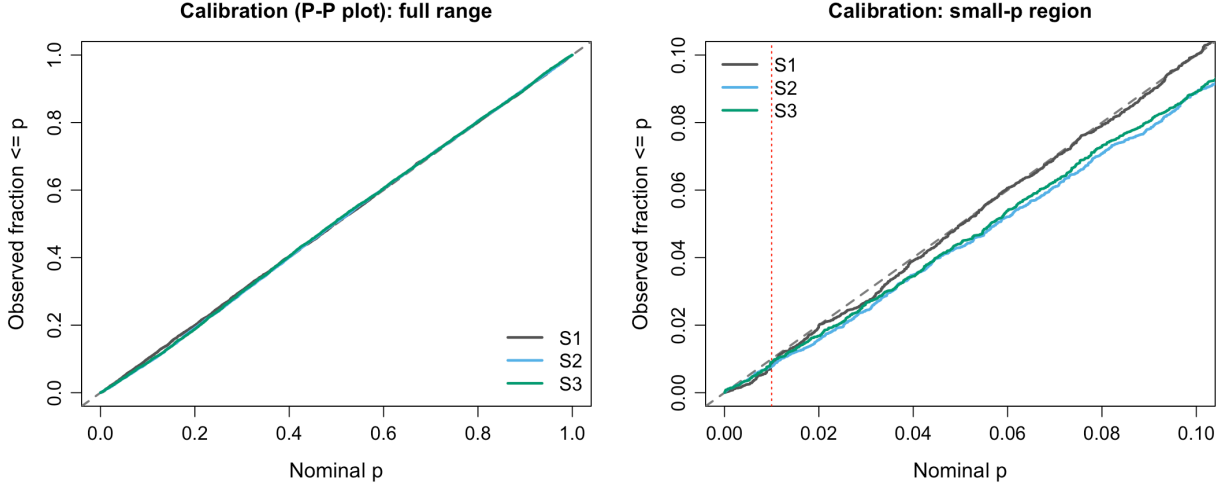
# Zoom into the testing-relevant small-p region
plot(NA, xlim = c(0, 0.1), ylim = c(0, 0.1),

```

```

    xlab = "Nominal p", ylab = "Observed fraction <= p",
    main = "Calibration: small-p region")
abline(0, 1, col = "grey50", lwd = 2, lty = 2)
abline(v = 0.01, col = "red", lwd = 1.2, lty = 3)
for (ti in 1:3) {
  ps <- sort(p_lists[[ti]])
  lines(ps, (seq_along(ps)) / length(ps), col = cols_b[ti], lwd = 2.5)
}
legend("topleft", bty = "n", cex = 1.1, lwd = 2.5, col = cols_b,
      legend = names(stat_list_b))

```



Result. All three statistics are calibrated: every realised false-positive rate sits at or below its nominal α , and the P–P curves track the diagonal. This is expected – empirical p-values are self-calibrating by construction, because each statistic is ranked against *its own* null, so the over-compression of S2/S3 cancels between the null and the test bins. The narrow histograms of S2/S3 therefore do **not** cause false positives; superficial non-normality was a red herring.

The check still discriminates, though. **S1 is the best calibrated** (KS $D \approx 0.009$, with FPR almost exactly 0.01 and 0.05), tracking the uniform diagonal nearly perfectly. S2 and S3 are slightly **conservative** – their realised FPR falls *below* α and their P–P curves bow under the diagonal – because the heavy aggregation discretises the statistic and inflates ties. Conservatism is safe but costs power: at a fixed α , a conservative test detects fewer true events. Calibration alone does not strictly eliminate any candidate, but it gives us a principled basis to choose: **S1 is both the best-calibrated (tightest to the uniform diagonal) and the simplest** – a single-bin statistic with no aggregation – while the conservatism of S2/S3 implies they would detect fewer true events at the same α . We therefore carry **S1** forward as our statistic for the genome-wide scan in Part C.

Part C - Computing P-values

Based on our conclusion in Part B, we proceed with **Statistic S1** (plain log) and **Strategy B** (local stability filter). We use the control bins identified by Strategy B to define our empirical null distribution. For every testable bin on the chromosome – one with reads in both the tumor and the matched healthy sample – we calculate a two-sided empirical p-value. The empirical null is summarised below: it is built from the Strategy-B control bins, is centred essentially at 0 (confirming the $\log_2$ ratio is unbiased under H_0), and is narrow enough to resolve very small p-values.


```

# Isolate the empirical null distribution values
null_stats <- df$S1[stratB & !is.na(df$S1)]
n_null     <- length(null_stats)
null_mean  <- mean(null_stats)
null_sd    <- sd(null_stats)

# Report the null summary as a clean table
null_summary <- data.frame(
  Quantity = c("Control bins (null size)", "Null mean (S1)", "Null SD (S1)",
    "Smallest attainable p-value"),
  Value = c(format(n_null, big.mark = ","),
    sprintf("%.5f", null_mean),
    sprintf("%.5f", null_sd),
    signif(1 / n_null, 2))
)
knitr::kable(null_summary, align = c("l", "r"),
  caption = "Empirical null distribution of S1 on the control bins.")

```

Table 2: Empirical null distribution of S1 on the control bins.

Quantity	Value
Control bins (null size)	30,959
Null mean (S1)	-0.00200
Null SD (S1)	0.16818
Smallest attainable p-value	3.2e-05

```

# Calculate empirical two-sided p-values.
# p_k = |{j in ctrl : |T_j| >= |T_k|}| / |ctrl|
# IMPORTANT: only test bins with reads in *both* samples (nz). A bin with zero
# coverage in a sample gets an eps-smoothed ratio far from 1 and would read as a
# huge "loss" -- e.g. the unmappable pericentromeric band -- so we leave such
# bins untested (p = NA) rather than calling them significant.
abs_null <- abs(null_stats)

df$p_val <- NA_real_
df$p_val[nz] <- sapply(abs(df$S1[nz]), function(x) {
  if (is.na(x)) return(NA_real_)
  sum(abs_null >= x) / n_null
})

# Prevent p=0 for FDR correction stability
min_p <- 1 / n_null
df$p_val[nz] <- pmax(df$p_val[nz], min_p)

# Preview the most extreme bins (smallest p-values), as a clean table
top_bins <- df[order(df$p_val), ][1:10, ]
top_tbl <- data.frame(
  `Position (Mb)` = round(top_bins$pos / 1e6, 3),
  `Tumor reads`   = top_bins$reads,
  `Ratio a-hat`   = round(top_bins$cn_2s, 3),
  `S1 = log2`     = round(top_bins$S1, 3),
  `p-value`       = signif(top_bins$p_val, 2),
  check.names = FALSE
)

```

```
)
knitr::kable(top_tbl, align = "r",
  caption = "Ten most extreme bins by empirical p-value (smallest p first).")
```

Table 3: Ten most extreme bins by empirical p-value (smallest p first).

Position (Mb)	Tumor reads	Ratio a-hat	S1 = log2	p-value
5.036	167	0.602	-0.732	3.2e-05
5.054	102	2.021	1.015	3.2e-05
5.524	132	2.135	1.094	3.2e-05
6.244	109	2.054	1.039	3.2e-05
6.291	353	1.637	0.711	3.2e-05
6.596	71	1.987	0.991	3.2e-05
7.256	165	1.679	0.748	3.2e-05
7.686	180	1.580	0.660	3.2e-05
7.791	230	1.642	0.715	3.2e-05
7.814	194	1.579	0.659	3.2e-05

Comparison with a parametric alternative

We could instead assume the statistic is Normal under H_0 and read p-values off the fitted curve: with μ_0, σ_0 estimated from the same control bins, the two-sided parametric p-value is $p_k = 2[1 - \Phi(|T_k - \mu_0|/\sigma_0)]$. It is worth asking how much this choice actually matters – whether the two approaches flag different bins.

```
# Parametric (Normal) p-values from the same control-bin mean/SD, for comparison.
z_score      <- (df$S1 - null_mean) / null_sd
df$p_val_param <- NA_real_
df$p_val_param[nz] <- 2 * pnorm(-abs(z_score[nz]))

# BH on both, then compare the significant calls
sig_emp      <- !is.na(df$p_val)      & p.adjust(df$p_val, method = "BH") < 0.01
sig_param    <- !is.na(df$p_val_param) & p.adjust(df$p_val_param, method = "BH") < 0.01

cmp_tbl <- data.frame(
  Quantity = c("Nominally significant (p < 0.01)",
    "Significant after BH (FDR < 0.01)",
    "Called by both",
    "Called by empirical only",
    "Called by parametric only"),
  Count = c(sprintf("%d / %d", sum(df$p_val < 0.01, na.rm = TRUE),
    sum(df$p_val_param < 0.01, na.rm = TRUE)),
    sprintf("%d / %d", sum(sig_emp), sum(sig_param)),
    sum(sig_emp & sig_param),
    sum(sig_emp & !sig_param),
    sum(sig_param & !sig_emp))
)
knitr::kable(cmp_tbl, align = c("l", "r"),
  col.names = c("Quantity", "Empirical / Parametric"),
  caption = "Bins flagged by the empirical vs parametric (Normal) p-values.")
```

Table 4: Bins flagged by the empirical vs parametric (Normal) p-values.

Quantity	Empirical / Parametric
Nominally significant ($p < 0.01$)	7334 / 7238
Significant after BH (FDR < 0.01)	2913 / 2340
Called by both	2340
Called by empirical only	573
Called by parametric only	0

```

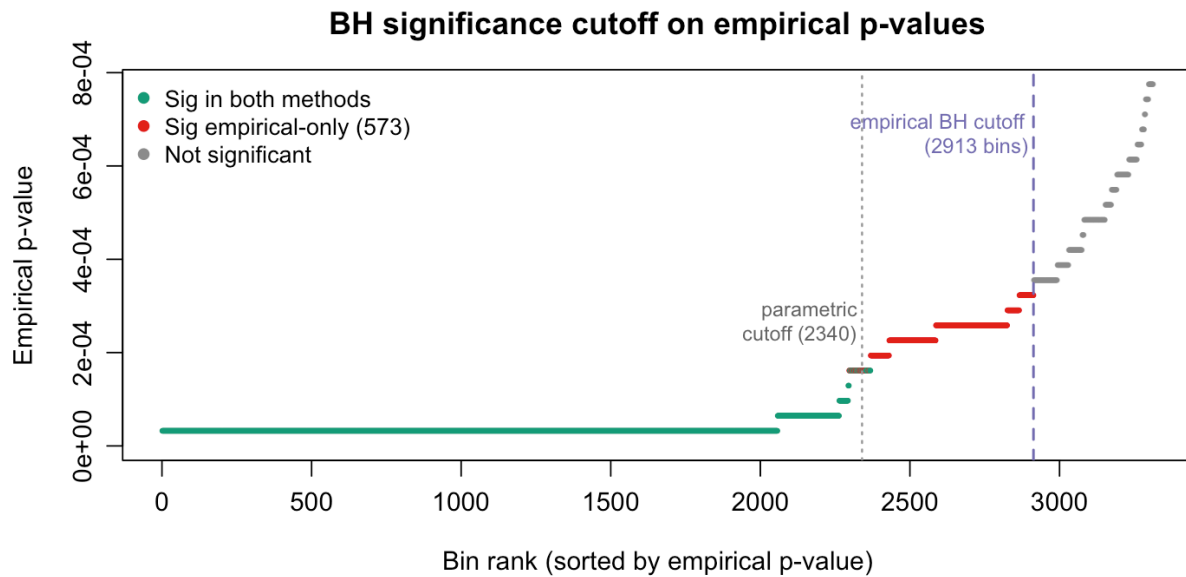
# Bins sorted by empirical p-value vs rank, with the BH significance cutoffs marked.
# Colour carries the empirical-vs-parametric difference: which significant bins
# both methods agree on, vs the ones only the empirical test keeps.
alpha <- 0.01
pv <- df$p_val[nz]; se <- sig_emp[nz]; sp <- sig_param[nz]
m <- length(pv)
ord <- order(pv)
pv_s <- pv[ord]; se_s <- se[ord]; sp_s <- sp[ord]
rank <- seq_len(m)

col_cls <- ifelse(se_s & sp_s, "both",
  ifelse(se_s & !sp_s, "emp_only", "none"))
pal <- c(both = "#1b9e77", emp_only = "#e31a1c", none = adjustcolor("grey55", 0.5))

nsig <- sum(se); nsig_par <- sum(sp)
xmax <- nsig + 400; sel <- rank <= xmax      # zoom to the decision region

par(mar = c(4.5, 4.7, 3, 1), mgp = c(2.7, 0.8, 0))
plot(rank[sel], pv_s[sel], pch = 20, cex = 0.5, col = pal[col_cls[sel]],
  xlab = "Bin rank (sorted by empirical p-value)", ylab = "Empirical p-value",
  main = "BH significance cutoff on empirical p-values", ylim = c(0, max(pv_s[sel])))
abline(v = nsig_par, col = "#999999", lty = 3, lwd = 1.3)
abline(v = nsig, col = "#7570b3", lty = 2, lwd = 1.6)
text(nsig, max(pv_s[sel]) * 0.92, sprintf("empirical BH cutoff\n(%d bins)", nsig),
  col = "#7570b3", adj = c(1.05, 1), cex = 0.8)
text(nsig_par, max(pv_s[sel]) * 0.40, sprintf("parametric\ncutoff (%d)", nsig_par),
  col = "#666666", adj = c(1.05, 1), cex = 0.78)
legend("topleft", bty = "n", pch = 20, pt.cex = 1.3, cex = 0.9,
  col = c("#1b9e77", "#e31a1c", "grey55"),
  legend = c("Sig in both methods", "Sig empirical-only (573)", "Not significant"))

```



The BH procedure keeps every bin up to the largest rank whose sorted p-value still satisfies the FDR criterion; the dashed line marks that empirical cutoff (2,913 bins) and the dotted line the parametric one (2,340). For the vast majority of bins the choice of p-value does not matter: the two methods agree on the 2,340 calls with the smallest p-values (green) and on the tens of thousands of clearly non-significant bins (grey). The disagreement is small and one-sided – the empirical test keeps 573 additional bins (red), while the parametric keeps none that the empirical misses, so the parametric calls are a strict subset. Those 573 red bins are the marginal ones lying between the two cutoffs: the more conservative Normal model drops them, costing power, with no compensating gain. In the opposite extreme it is over-confident, assigning the single most extreme bin $p \approx 10^{-133}$ – a precision the \$31,000 control observations cannot support, whereas the empirical p-value bottoms out at its resolution floor $1/m \approx 3.2 \times 10^{-5}$. Since the empirical null also requires no distributional assumption, we retain it for the genome-wide scan.

Part D - FDR Correction and Segmentation

At the nominal level $\alpha = 0.01$, before any correction, a large number of bins are flagged (the uncorrected count is reported below, answering part (c) – but with tens of thousands of bins tested, roughly $\alpha \times N$ of those are expected false positives purely by chance, so the raw count is not trustworthy.

The property the correction preserves (part d). We use the **Benjamini–Hochberg (BH)** procedure, which controls the **False Discovery Rate (FDR)** – the *expected proportion of false positives among the bins we call significant*. Setting the BH target to 0.01 guarantees that, on average, at most 1% of our reported significant bins are false discoveries. This is deliberately different from Bonferroni, which controls the **family-wise error rate** (the probability of *even one* false positive anywhere) and would be far too strict here: with \$85,000 tests Bonferroni’s per-test threshold collapses to $\approx 10^{-7}$ and would discard almost all of the true, weaker copy-number events. Because we are screening a whole chromosome for *many* real alterations rather than guarding against a single false call, FDR control is the appropriate trade-off – it tolerates a small, known fraction of false positives in exchange for much greater power to detect true events. Adjacent significant windows are then grouped into continuous genomic segments (events).

```
alpha <- 0.01

# Apply Benjamini-Hochberg FDR correction
df$p_adj <- p.adjust(df$p_val, method = "BH")
```

```

df$is_sig <- !is.na(df$p_adj) & df$p_adj < alpha

# Determine the type of change (Gain vs Loss)
# Since S1 = log2(a_hat), positive is gain, negative is loss.
df$change_type <- "Normal"
df$change_type[df$is_sig & df$S1 > 0] <- "Gain"
df$change_type[df$is_sig & df$S1 < 0] <- "Loss"

# Headline counts: tested bins, nominal hits (part D), and post-correction calls
n_tested <- sum(!is.na(df$p_val))
n_raw_sig <- sum(df$p_val < alpha, na.rm = TRUE) # before correction (spec part D)
n_sig <- sum(df$is_sig, na.rm = TRUE)
count_tbl <- data.frame(
  Quantity = c("Testable bins (reads in both samples)",
               sprintf("Nominally significant (p < %.2f, uncorrected)", alpha),
               sprintf("Significant after BH (FDR < %.2f)", alpha),
               " - of which Gains (ratio > 1)",
               " - of which Losses (ratio < 1)"),
  Count = c(n_tested, n_raw_sig, n_sig,
            sum(df$change_type == "Gain"), sum(df$change_type == "Loss"))
)
knitr::kable(count_tbl, align = c("l", "r"),
              caption = "Significant-bin counts before and after multiple-testing
              ↪ correction.")

```

Table 5: Significant-bin counts before and after multiple-testing correction.

Quantity	Count
Testable bins (reads in both samples)	85010
Nominally significant (p < 0.01, uncorrected)	7334
Significant after BH (FDR < 0.01)	2913
- of which Gains (ratio > 1)	1600
- of which Losses (ratio < 1)	1313

```

# Group adjacent significant bins into segments using rle
rle_res <- rle(df$change_type)
end_idx <- cumsum(rle_res$lengths)
start_idx <- c(1, head(end_idx, -1) + 1)

segments <- data.table(
  change_type = rle_res$values,
  start_idx = start_idx,
  end_idx = end_idx,
  n_bins = rle_res$lengths
)

# Filter for significant segments only
sig_segments <- segments[change_type != "Normal"]

# Calculate genomic coordinates and summary stats for each segment.
# Report the two-sample ratio cn_2s (= tumor/healthy) -- that is the quantity the
# whole test is built on. The single-sample tumor cn ignores the healthy baseline

```

```

# and can disagree in sign with the event call.
sig_segments[, `:=`(
  start_mb = df$pos[start_idx] / 1e6,
  end_mb   = df$pos[end_idx] / 1e6,
  size_kb  = (df$pos[end_idx] - df$pos[start_idx] + BIN) / 1e3,
  mean_ratio = mapply(function(s, e) mean(df$cn_2s[s:e], na.rm = TRUE), start_idx,
    ↪ end_idx),
  min_padj = mapply(function(s, e) min(df$p_adj[s:e], na.rm = TRUE), start_idx, end_idx)
)]

# Clean up table for display, filtering for segments > 10kb to remove single-bin noise
segments_display <- sig_segments[size_kb > 10, .(
  Event = change_type,
  Start_Mb = round(start_mb, 2),
  End_Mb = round(end_mb, 2),
  Size_kb = round(size_kb, 1),
  Bins = n_bins,
  Mean_ratio = round(mean_ratio, 3),
  Min_FDR = signif(min_padj, 3)
)]

knitr::kable(segments_display, caption = "Significant copy-number alteration segments
  ↪ (FDR < 0.01, size > 10 kb).")

```

Table 6: Significant copy-number alteration segments (FDR < 0.01, size > 10 kb).

Event	Start_Mb	End_Mb	Size_kb	Bins	Mean_ratio	Min_FDR
Loss	28.57	28.63	60.0	24	0.537	0.00133
Loss	141.54	141.55	12.5	5	0.588	0.00133
Loss	141.56	141.57	12.5	5	0.570	0.00133
Loss	141.57	141.60	35.0	14	0.528	0.00133
Loss	141.61	141.62	12.5	5	0.552	0.00133
Loss	141.73	141.79	62.5	25	0.460	0.00133
Loss	141.80	141.82	20.0	8	0.500	0.00133
Loss	141.82	141.84	15.0	6	0.439	0.00133
Loss	141.84	141.86	22.5	9	0.534	0.00133
Loss	141.88	141.90	22.5	9	0.521	0.00133
Loss	142.12	142.13	12.5	5	0.510	0.00133

Part E - Visualization

To visualise the results of our copy-number analysis, we plot the $S1$ statistic ($\log_2(\hat{a}_k)$) across the chromosome, highlighting the regions identified as statistically significant amplifications (Gain, CN > 1) and deletions (Loss, CN < 1).

We provide two views: a static genome-wide plot (below) that renders in any output format, and an interactive Plotly version (rendered only in the HTML build) that supports zoom, pan and per-bin hover – the “movable graph” the task allows as an alternative to a region list.

```

# Static genome-wide view -- always renders (PDF and HTML).
set.seed(42)
norm_idx <- which(df$change_type == "Normal" & !is.na(df$S1))

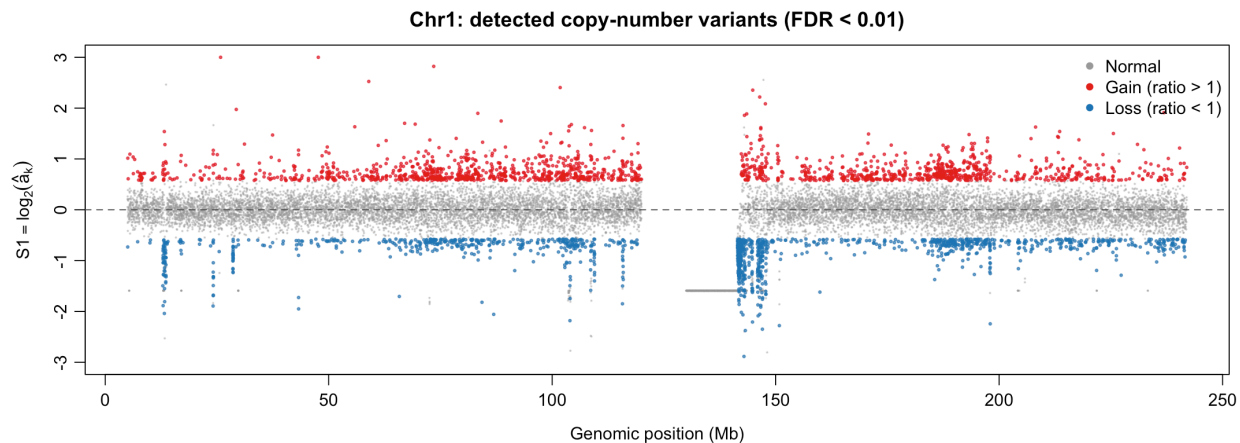
```

```

norm_s   <- sample(norm_idx, min(15000, length(norm_idx)))
gain_idx <- which(df$change_type == "Gain")
loss_idx <- which(df$change_type == "Loss")

par(mar = c(4.2, 4, 2.5, 1), mgp = c(2.3, 0.7, 0))
plot(df$pos[norm_s] / 1e6, pmin(pmax(df$S1[norm_s], -3), 3),
     pch = 20, cex = 0.25, col = adjustcolor("grey60", 0.35),
     xlab = "Genomic position (Mb)", ylab = expression(S1 == log[2](hat(a)[k])),
     main = "Chr1: detected copy-number variants (FDR < 0.01)",
     ylim = c(-3, 3), xlim = range(df$pos / 1e6))
abline(h = 0, col = "grey30", lty = 2)
points(df$pos[loss_idx] / 1e6, pmin(pmax(df$S1[loss_idx], -3), 3),
       pch = 20, cex = 0.5, col = adjustcolor("#1f78b4", 0.7))
points(df$pos[gain_idx] / 1e6, pmin(pmax(df$S1[gain_idx], -3), 3),
       pch = 20, cex = 0.5, col = adjustcolor("#e31a1c", 0.7))
legend("topright", bty = "n", pch = 20, pt.cex = 1.2,
      col = c("grey60", "#e31a1c", "#1f78b4"),
      legend = c("Normal", "Gain (ratio > 1)", "Loss (ratio < 1)"))

```



It is also worth seeing the **raw tumor read counts** directly, with each bin coloured by its anomaly call. This shows the detection acting on the actual coverage: significant losses sit in the low read-count bins and gains in the high ones, while the grey background stays near the chromosome-wide median (~216 reads / 2.5 kb bin).

```

# Raw tumor read counts per bin, coloured by CNV call. Pure scatter: no lines,
# so the centromere gap stays empty (no interpolation across it).
ycap <- quantile(df$reads[nz], 0.995) # cap y so a few huge bins don't dominate
x_mb <- df$pos / 1e6
y <- pmin(df$reads, ycap)

# Draw Normal first, then anomalies on top so the calls are not buried
ord <- order(df$change_type != "Normal")
cols <- c(Normal = adjustcolor("grey65", 0.35),
         Gain = adjustcolor("#e31a1c", 0.8),
         Loss = adjustcolor("#1f78b4", 0.8))

par(mar = c(4.2, 4, 2.5, 1), mgp = c(2.3, 0.7, 0))
plot(x_mb[ord], y[ord], pch = 20,
     cex = ifelse(df$change_type[ord] == "Normal", 0.2, 0.5),

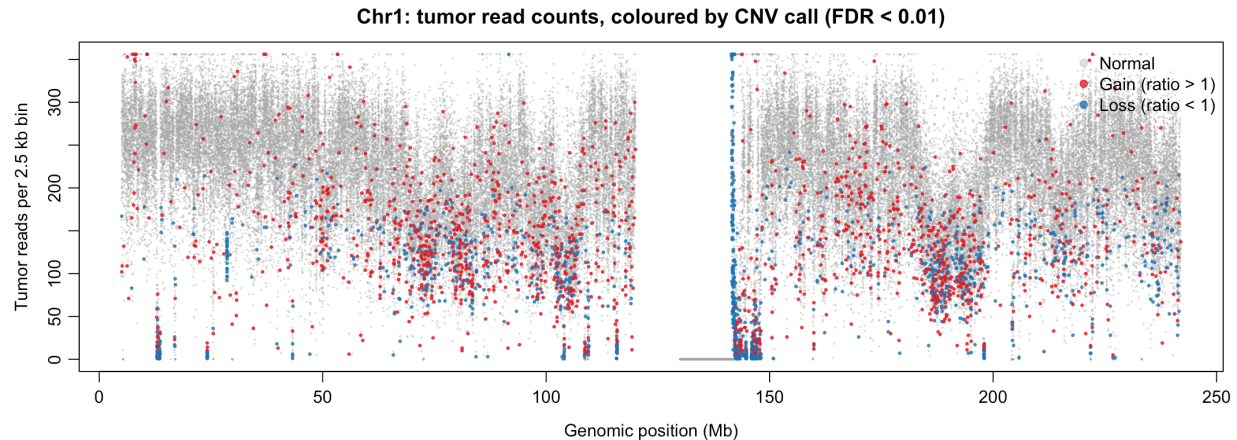
```



```

col = cols[df$change_type[ord]],
xlab = "Genomic position (Mb)", ylab = "Tumor reads per 2.5 kb bin",
main = "Chr1: tumor read counts, coloured by CNV call (FDR < 0.01)",
ylim = c(0, ycap), xlim = range(x_mb))
legend("topright", bty = "n", pch = 20, pt.cex = 1.4,
col = cols[c("Normal", "Gain", "Loss")],
legend = c("Normal", "Gain (ratio > 1)", "Loss (ratio < 1)"))

```



The interactive version below downsamples the “Normal” bins (keeping all significant ones) so the widget stays responsive. It is always exported to a self-contained `AnomalyDetections.html` file (which can be hosted and linked from a slide deck), and is shown inline only in the HTML build.

```

if (!require('plotly')) { install.packages('plotly'); library('plotly') }
if (!require('htmlwidgets')) { install.packages('htmlwidgets'); library('htmlwidgets') }

# Downsample 'Normal' bins to keep the Plotly interactive plot responsive
set.seed(42)
normal_bins <- which(df$change_type == "Normal" & !is.na(df$S1))
sampled_normal <- sample(normal_bins, min(10000, length(normal_bins)))
sig_bins <- which(df$change_type != "Normal" & !is.na(df$S1))

# Combine sampled normal bins and all significant bins
plot_df <- df[c(sampled_normal, sig_bins), ]
# Reorder by position so the hover behaves nicely
plot_df <- plot_df[order(plot_df$pos), ]

fig <- plot_ly(
  data = plot_df,
  x = ~pos / 1e6,
  y = ~S1,
  type = 'scatter',
  mode = 'markers',
  color = ~change_type,
  colors = c("Normal" = "grey", "Gain" = "red", "Loss" = "blue"),
  marker = list(size = 3, opacity = 0.5),
  text = ~paste("Position:", round(pos / 1e6, 3), "Mb<br>S1 Stat:", round(S1, 4),
    ↪ "<br>Status:", change_type),
  hoverinfo = "text"
) %>%

```



```

layout(
  title = "Chromosome 1: Detected Copy Number Variations",
  xaxis = list(title = "Genomic Position (Mb)",
  yaxis = list(title = "S1 Statistic (log(a-hat))",
  legend = list(title = list(text = "<b> Status </b>"))
)

# Always write the standalone interactive file (for hosting / slide linking)
htmlwidgets::saveWidget(fig, "AnomalyDetections.html", selfcontained = TRUE)

# Show inline only in HTML output; the PDF uses the static plot above
if (knitr::is_html_output()) fig else knitr::asis_output("")

```

These plots bring together our robust statistic mapping, the empirical null calibration, and the Benjamini-Hochberg FDR filtering across the entirety of Chromosome 1.

Several features are visible across the chromosome:

- **The centromere gap.** The clear window between roughly 120 Mb and 130 Mb is the centromeric region, excluded during binning because of known reference-mapping assembly gaps. We only test bins with reads in *both* samples, so the unmappable pericentromeric flanks are left untested rather than mis-called as losses.
- **The background envelope.** The dense band of grey markers around $S1 = 0$ is the copy-number-neutral background (H_0). Because the empirical null is built from the locally stable control bins (Strategy B), the bulk of both arms sits flat near zero, as expected.
- **Significant losses (blue).** The clearest focal deletion is near 28.5 Mb (mean ratio ≈ 0.54 , a roughly two-fold loss), and a cluster of significant losses appears around 141–142 Mb. These are evaluated on the two-sample ratio, so they reflect tumor-specific change relative to the matched healthy sample, not absolute tumor coverage.
- **Significant gains (red).** Bins rising above the baseline envelope are candidate tumor-specific amplifications; the calls are more scattered and focal than the losses.

Because we test the single-bin statistic $S1$, individual calls are at fine resolution; the `rle` segmentation then groups runs of same-direction significant bins into the events listed in the table above. The size filter (> 10 kb) in that table suppresses isolated single-bin calls, while the plots show every significant bin.

Part F – Local Zoom-In Plots

We zoom into four candidate areas (75 Mb, 100 Mb, 145 Mb, 190 Mb), comparing raw tumor read counts (coloured by CNV call) against the matched healthy sample. A 51-bin rolling median (≈ 127.5 kb) is overlaid for each sample to reveal regional trends through bin-level noise.

```

if (!require('ggplot2')) { install.packages('ggplot2'); library('ggplot2') }
if (!require('dplyr')) { install.packages('dplyr'); library('dplyr') }
if (!require('patchwork')) { install.packages('patchwork'); library('patchwork') }
if (!require('zoo')) { install.packages('zoo'); library('zoo') }

cnv_zoom_df <- data.frame(
  pos_mb = df$pos / 1e6,
  tumor_reads = df$reads,
  healthy_reads = df_h$reads,
  S1 = df$S1,
  status = factor(df$change_type, levels = c("Normal", "Gain", "Loss"))
)

```

```

k_roll <- 51
cnv_zoom_df <- cnv_zoom_df %>%
  mutate(
    tumor_rollmed = rollmedian(tumor_reads, k = k_roll, fill = NA, align = "center"),
    healthy_rollmed = rollmedian(healthy_reads, k = k_roll, fill = NA, align = "center"),
    S1_rollmed = rollmedian(S1, k = k_roll, fill = NA, align = "center")
  )

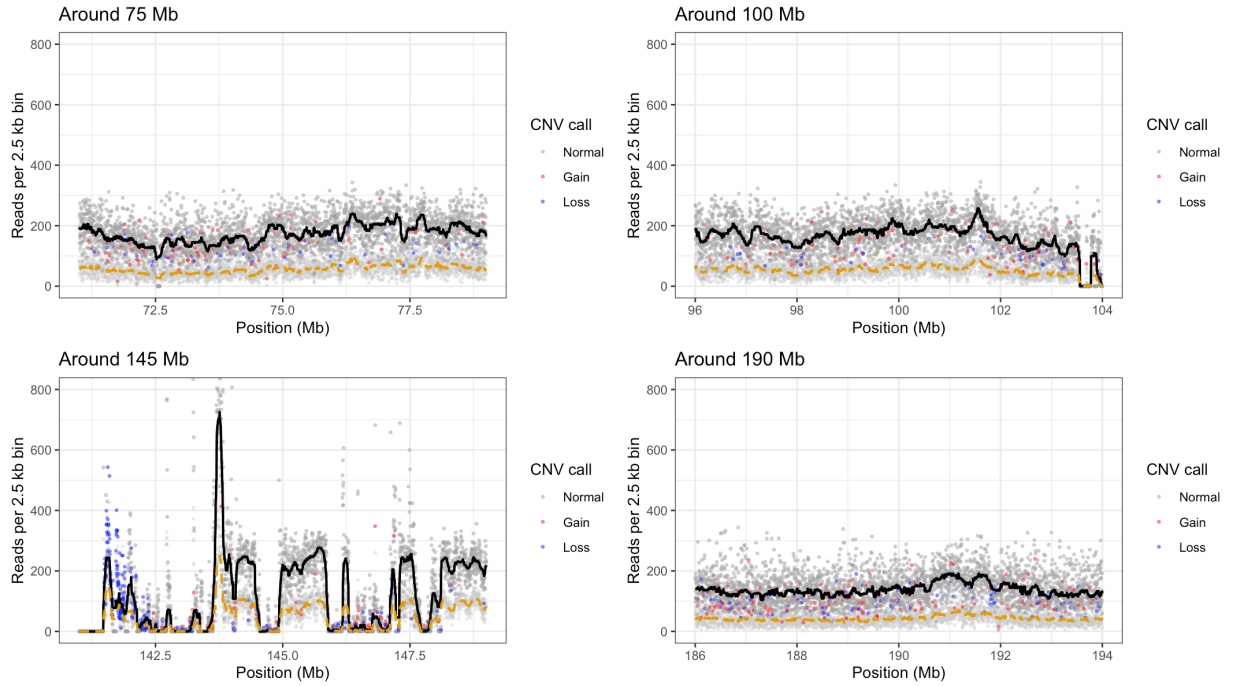
plot_reads_zoom <- function(zdf, center_mb, window_mb = 8, title = NULL) {
  if (is.null(title)) title <- paste0("Around ", center_mb, " Mb")
  region <- zdf %>%
    filter(pos_mb >= center_mb - window_mb / 2,
           pos_mb <= center_mb + window_mb / 2)
  pal <- c(Normal = "grey65", Gain = "red", Loss = "blue")
  ggplot(region, aes(x = pos_mb)) +
    geom_point(aes(y = healthy_reads), color = "grey80", alpha = 0.30, size = 0.5) +
    geom_point(aes(y = tumor_reads, color = status), alpha = 0.45, size = 0.6) +
    geom_line(aes(y = tumor_rollmed), color = "black", linewidth = 0.8) +
    geom_line(aes(y = healthy_rollmed), color = "#E69F00", linewidth = 0.8, linetype =
      ↪ "dashed") +
    scale_color_manual(values = pal) +
    coord_cartesian(ylim = c(0, 800)) +
    labs(
      title = title,
      x = "Position (Mb)",
      y = "Reads per 2.5 kb bin",
      color = "CNV call"
    ) +
    theme_bw()
}

r75 <- plot_reads_zoom(cnv_zoom_df, 75, 8, "Around 75 Mb")
r100 <- plot_reads_zoom(cnv_zoom_df, 100, 8, "Around 100 Mb")
r145 <- plot_reads_zoom(cnv_zoom_df, 145, 8, "Around 145 Mb")
r190 <- plot_reads_zoom(cnv_zoom_df, 190, 8, "Around 190 Mb")

(r75 | r100) / (r145 | r190) +
  plot_annotation(subtitle = "Black line = tumor rolling median | Orange dashed =
    ↪ healthy rolling median | Points coloured by CNV call")

```

Black line = tumor rolling median | Orange dashed = healthy rolling median | Points coloured by CNV call



External sources statement

We made extensive use of AI assistants throughout this lab, in several distinct roles:

- **Background research.** We used ChatGPT and Claude to survey the copy-number-detection literature and to suggest candidate test statistics, which led us to consider robust window aggregates (trimmed mean, Huber location) alongside the plain per-bin statistic.
- **Method exploration.** We used Claude to prototype and compare several negative-control selection strategies (e.g. a MAD-3 outlier filter versus the local-stability filter we ultimately adopted), and to design the split-half calibration check used to choose the final statistic.
- **Code.** We used Claude to write and debug much of the R code – the empirical p-value computation, the Benjamini–Hochberg correction and `rle` segmentation, and the parametric-vs-empirical comparison – including catching and fixing correctness bugs (for example, restricting the test to bins with reads in both samples).
- **Visualisation.** We used Claude to build and iterate on the figures, including the genome-wide scatter plots, the calibration P–P plots, the BH-cutoff comparison plot, and the interactive Plotly widget exported as a standalone HTML file.
- **Writing.** We used Claude to help structure this document and tighten its prose.

All statistical choices (window width, stability thresholds, the FDR target), the interpretation of the results, and the final decisions on what to include remain our own; we reviewed and verified the AI-generated code and text throughout.